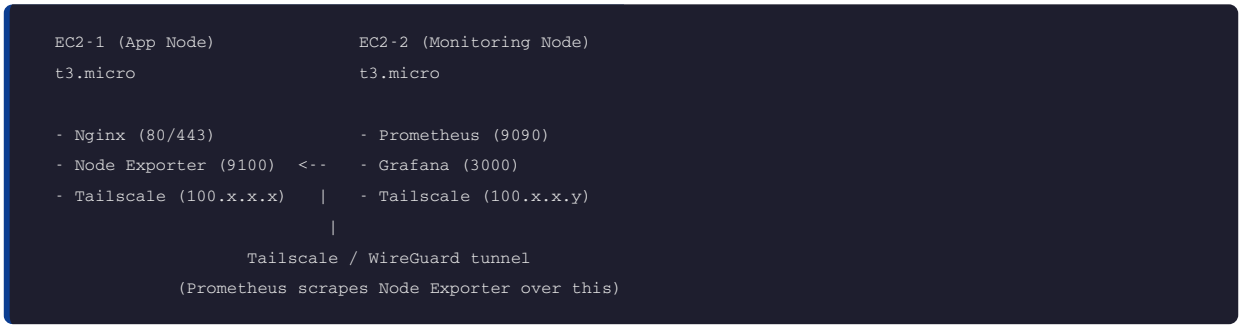
Cross-Server Monitoring with Tailscale, Prometheus and Grafana

Cross-Server Monitoring with Tailscale, Prometheus & Grafana

A zero-trust monitoring setup where two AWS EC2 instances communicate over a private Tailscale mesh network instead of the public internet. One instance runs an application (Nginx + Node Exporter); the other runs the monitoring stack (Prometheus + Grafana) that scrapes it.

Architecture



Node Exporter's metrics port (9100) and Prometheus's port (9090) are **never opened in the AWS Security Groups**. Prometheus reaches Node Exporter exclusively via its Tailscale IP, so the only way onto that traffic path is by being an authenticated member of the same tailnet.

Why Tailscale Instead of Opening Ports Publicly

Security Groups filter traffic based on the public/private IP and port hitting the instance's normal network interface (ENI). Tailscale traffic arrives as encrypted WireGuard UDP packets (port 41641), which the Security Group can't inspect. The Tailscale daemon decrypts this locally and injects it into a virtual network interface called `tailscale0` — from the OS's perspective, this looks like normal traffic on that interface, but it never touches port 9100/9090 on the SG-monitored ENI.

Practical result: as long as you never open 9100 or 9090 in your Security Groups, only devices authenticated to your tailnet can reach those services — regardless of what's technically "possible" over the public internet.

Prerequisites

- Two AWS EC2 instances (t3.micro is sufficient — see Resource Notes)
- Ubuntu 22.04 LTS (or similar) on both
- A free Tailscale account
- Basic familiarity with SSH and systemd

Phase 1: Provision the EC2 Instances

1. **Launch EC2-1 (App node)**
   - AMI: Ubuntu 22.04 LTS
   - Type: t3.micro
   - Security Group: allow inbound `22` (SSH, your IP only), `80` / `443` (public, for Nginx)
   - Do **not** open port `9100`
2. **Launch EC2-2 (Monitoring node)**
   - Same AMI/type
   - Security Group: allow inbound `22` (SSH, your IP only)
   - Optionally allow `3000` (Grafana) restricted to your IP — or skip entirely and access Grafana purely over Tailscale (recommended)
   - Do **not** open port `9090`

Phase 2: Install Tailscale on Both Nodes

Run this on **both** EC2-1 and EC2-2:

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

This prints a login URL — authenticate both machines to the **same** Tailscale account so they join the same tailnet.

Get each machine's Tailscale IP:

```
tailscale ip -4
```

Write these down. In this guide: - `TS_IP_APP` = EC2-1's Tailscale IP - `TS_IP_MON` = EC2-2's Tailscale IP

> ⚠ **Double-check which IP belongs to which machine.** Mixing these up is the single most common mistake in this setup (see Troubleshooting) — it causes Prometheus to scrape itself instead of the app node.

Phase 3: Set Up EC2-1 (Nginx + Node Exporter)

3.1 Install Nginx

```
sudo apt update && sudo apt install -y nginx
sudo systemctl enable --now nginx
```

3.2 Install Node Exporter

Check the latest version first — **do not use wildcards in `wget` URLs**, they aren't expanded and will 404:

```
NE_VERSION=$(curl -s https://api.github.com/repos/prometheus/node_exporter/releases/latest \
  | grep '"tag_name"' \
  | sed -E 's/.*"v([^"]+)".*/\1/')

cd /tmp

wget https://github.com/prometheus/node_exporter/releases/download/v${NE_VERSION}/\
node_exporter-${NE_VERSION}.linux-amd64.tar.gz

tar xvf node_exporter-${NE_VERSION}.linux-amd64.tar.gz
```

```
sudo mv node_exporter-${NE_VERSION}.linux-amd64/node_exporter /usr/local/bin/
```

3.3 Create a Dedicated User

Running services as a non-login system user limits blast radius if the service is ever compromised:

```
sudo useradd --no-create-home --shell /usr/sbin/nologin node_exporter
sudo chown node_exporter:node_exporter /usr/local/bin/node_exporter
```

3.4 Create the systemd Service

```
sudo nano /etc/systemd/system/node_exporter.service
```

```
[Unit]
Description=Prometheus Node Exporter
Wants=network-online.target
After=network-online.target

[Service]
User=node_exporter
Group=node_exporter
Type=simple
ExecStart=/usr/local/bin/node_exporter

[Install]
WantedBy=multi-user.target
```

Enable and start:

```
sudo systemctl daemon-reload
sudo systemctl enable --now node_exporter
sudo systemctl status node_exporter
```

3.5 Verify Locally

```
curl localhost:9100/metrics
```

You should see a large block of metrics such as `node_cpu_seconds_total` and `node_memory_MemAvailable_bytes`.

3.6 Note the Firewall Decision

Node Exporter's port 9100 is already unreachable from the public internet because it's never opened in the Security Group. Traffic arriving via `tailscale0` bypasses the Security Group entirely (see Why Tailscale), so `ufw` **is optional defense-in-depth here, not a requirement** — it only matters if you want to guard against future Security Group misconfiguration or other devices inside the same tailnet/VPC. This project relies on Tailscale's own authentication as the real gatekeeper.

If you want the extra layer anyway:

```
sudo apt install -y ufw
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow in on tailscale0
sudo ufw enable
```

Phase 4: Set Up EC2-2 (Prometheus + Grafana)

4.1 Install Prometheus

Check the latest version and use the exact version number (again, no wildcards):

```
curl -s https://api.github.com/repos/prometheus/prometheus/releases/latest | grep tag_name
```

```
cd /tmp
wget https://github.com/prometheus/prometheus/releases/download/vX.X.X/\
prometheus-X.X.X.linux-amd64.tar.gz
tar xvf prometheus-X.X.X.linux-amd64.tar.gz
cd prometheus-X.X.X.linux-amd64
```

> **Note:** Prometheus v3.x removed the old `consoles` / `console_libraries` directories that appeared in older tutorials (part of the legacy web UI). If they're not in your extracted folder, that's expected for v3.x — skip moving them.

4.2 Move Binaries into Place

```
sudo mkdir -p /etc/prometheus /var/lib/prometheus
sudo mv prometheus /usr/local/bin/
sudo mv promtool /usr/local/bin/
```

4.3 Create a Dedicated User

```
sudo useradd --no-create-home --shell /usr/sbin/nologin prometheus
sudo chown -R prometheus:prometheus /etc/prometheus /var/lib/prometheus
sudo chown prometheus:prometheus /usr/local/bin/prometheus /usr/local/bin/promtool
```

4.4 Write the Prometheus Config

```
sudo nano /etc/prometheus/prometheus.yml
```

```
global:
  scrape_interval: 30s

scrape_configs:
  - job_name: 'node_exporter_app'
    static_configs:
      - targets: ['TS_IP_APP:9100']   # EC2-1's Tailscale IP — NOT EC2-2's own IP
```

> ⚠ **Triple-check this IP.** It must be EC2-1's Tailscale IP, not EC2-2's own. Run `tailscale ip -4` **on EC2-1**, not EC2-2, to get this value. See Troubleshooting for what happens if you get this wrong.

Set ownership:

```
sudo chown prometheus:prometheus /etc/prometheus/prometheus.yml
```

4.5 Create the systemd Service

```
sudo nano /etc/systemd/system/prometheus.service
```

```
[Unit]
Description=Prometheus
Wants=network-online.target
After=network-online.target
```

```
[Service]
User=prometheus
Group=prometheus
Type=simple
ExecStart=/usr/local/bin/prometheus \
  --config.file=/etc/prometheus/prometheus.yml \
  --storage.tsdb.path=/var/lib/prometheus \
  --storage.tsdb.retention.time=7d \
  --web.listen-address=0.0.0.0:9090

[Install]
WantedBy=multi-user.target
```

The `--storage.tsdb.retention.time=7d` flag keeps the time-series database small — important for the memory-constrained t3.micro (default retention is 15 days).

Enable and start:

```
sudo systemctl daemon-reload
sudo systemctl enable --now prometheus
sudo systemctl status prometheus
```

4.6 Verify the Scrape Target Is Reachable

From EC2-2, confirm Tailscale connectivity to EC2-1 directly:

```
curl -s http://TS_IP_APP:9100/metrics | head
```

Then check Prometheus's own view of the target:

```
curl -s http://localhost:9090/api/v1/targets | python3 -m json.tool
```

Look for `"health": "up"`. If it instead shows `"health": "down"` or the `up` metric evaluates to `0`, see Troubleshooting.

4.7 Install Grafana

```
sudo apt-get install -y apt-transport-https software-properties-common wget
sudo mkdir -p /etc/apt/keyrings/
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee /etc/apt/keyrings/grafana.gpg > /dev/null
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo tee -a
/etc/apt/sources.list.d/grafana.list
sudo apt-get update
sudo apt-get install -y grafana
```

> This uses the current `keyrings`-based method rather than the deprecated `apt-key add` approach seen in older tutorials.

4.8 Enable and Start Grafana

```
sudo systemctl daemon-reload
sudo systemctl enable --now grafana-server
sudo systemctl status grafana-server
curl -s http://localhost:3000/api/health
```

4.9 Access Grafana

- **Via Tailscale (recommended):** install Tailscale on your laptop too, join the same tailnet, then visit `http://TS_IP_MON:3000`
- **Via public IP:** only if you opened port 3000 in the Security Group, visit `http://<EC2-2_PUBLIC_IP>:3000`

Default login is `admin` / `admin` — you'll be forced to set a new password on first login.

Phase 5: Connect Grafana to Prometheus

1. **Connections → Data sources → Add data source → Prometheus**
2. Prometheus server URL: `http://localhost:9090` (same instance, so `localhost` is correct — no Tailscale IP needed here)
3. **Save & test** — should return "Successfully queried the Prometheus API"

Phase 6: Import the Node Exporter Dashboard

1. **Dashboards → New → Import**
2. Dashboard ID: `1860` (Node Exporter Full — community-maintained, works with current node_exporter metric names)
3. Select your Prometheus data source
4. **Import**

Check the **Job** and **Instance** dropdowns at the top of the dashboard — they should show `node_exporter_app` and `TS_IP_APP:9100`. Set the time range to **Last 15 minutes** and confirm CPU/memory/disk/network panels populate with live, non-zero values.

Resource Notes for t3.micro

Two t3.micro instances (1 vCPU, 1 GB RAM each, burstable) are sufficient for this project at demo/portfolio scale:

Component	Approx. RAM
Nginx	5–10 MB
Node Exporter	15–20 MB
Prometheus (7-day retention, 1 target)	150–250 MB
Grafana	100–150 MB
Tailscale daemon (each node)	10–15 MB

EC2-2 (Prometheus + Grafana together) runs closest to the 1 GB ceiling. **Add a swap file as a safety margin** — without it, a memory spike can cause the OOM killer to silently terminate Prometheus or Grafana:

```
sudo fallocate -l 1G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
free -h
```

If a service mysteriously stops running with no obvious cause, check for an OOM kill before anything else:

```
sudo systemctl status prometheus   # or grafana-server
dmesg | grep -i "out of memory" | tail -5
```

Troubleshooting

`wget`: `Cannot open: No such file or directory` after using a wildcard URL

`wget` does not expand `*` like a shell glob — it treats it as a literal character and 404s. Always resolve the exact version number first (via the GitHub releases API) and substitute it into the URL directly. See the dynamic version-lookup snippets in Phase 3.2 and Phase 4.1.

Grafana dashboard shows "No data" and the Job/Instance dropdowns are empty

This is a strong sign the underlying Prometheus scrape is failing (not a Grafana display bug). Before touching Grafana, check Prometheus directly:

```
curl -s http://localhost:9090/api/v1/query?query=up | python3 -m json.tool
```

- `"value": [..., "1"]` → scrape is healthy; the problem is in Grafana's data source binding — check **Connections → Data**

sources → Prometheus → Save & test, and confirm the dashboard's variables are bound to the correct data source.
- `"value": [..., "0"]` → the scrape itself is failing. Move to the next issue.

`up` returns `0`, or two different instance IPs show up

This almost always means the wrong Tailscale IP was entered in `prometheus.yml` — commonly, EC2-2's own Tailscale IP was pasted instead of EC2-1's. Prometheus then tries to scrape port 9100 on itself, where nothing is listening.

Fix: 1. Run `tailscale ip -4` **on EC2-1 specifically** and confirm the value. 2. Edit `/etc/prometheus/prometheus.yml` and set the `targets` list to that single, correct IP — remove any old/incorrect entries. 3. Restart: `sudo systemctl restart prometheus` 4. Verify with `curl -s http://localhost:9090/api/v1/targets | python3 -m json.tool` — this reflects the **current config**, unlike `/api/v1/query?query=up`, which can still return old, stale time-series points from before the fix even after the config is corrected. Old stale series are harmless and will naturally age out based on your retention window (7 days in this setup).

A service that was running has silently stopped

Check for an OOM kill first (see Resource Notes), then check the systemd journal for the real error:

```
sudo journalctl -u prometheus -n 50 --no-pager
```

`curl` command appears to hang or fail with a malformed URL

Double-check the URL includes `//` after the scheme — `http:100.121.170.83:9100` (missing slashes) is a common typo and will not behave like `http://100.121.170.83:9100`.

Security Notes

- Neither Node Exporter (9100) nor Prometheus (9090) is ever exposed in a Security Group — all scrape traffic travels exclusively over the Tailscale-encrypted tunnel.
- Both services run under dedicated, non-login system users rather than root or the default `ubuntu` user.
- Grafana is best accessed over Tailscale as well, avoiding any public exposure of port 3000.
- For further hardening, a Tailscale ACL policy can restrict which tailnet devices are permitted to reach port 9100 on the app node, rather than relying on implicit trust of all tailnet members.

Result

```
Nginx + Node Exporter (EC2-1)  ──over Tailscale──►  Prometheus + Grafana (EC2-2)
```

A fully working, privately-networked monitoring pipeline with no monitoring ports exposed to the public internet, running comfortably on two AWS free-tier-eligible t3.micro instances.